Symmetric Cell Search for Probing Security

How the verifier works, from the netlist to the verdict

Working notes for the `engine-speed` branch

31 October 2026

Abstract

These notes describe, from start to end, the algorithm that the Lean tool uses to decide d-probing security of a masked Boolean circuit. A rewrite engine certifies single probe sets and returns a *coupling*; the coupling extends a certified probe set into a large *safe set*; a *cell search* covers the space of all probe sets of size d with few engine calls by remembering every safe set; and the circuit's *automorphisms* are used twice — to start the search from cells that contain one representative of every orbit of probe sets (*orbital branching*), and to settle a probe set whenever a symmetric image of a stored safe set already contains it (*dominance detection*). Every step is stated with its soundness argument and with the Lean functions that implement it. Section 8 builds the orbital-branching shape of the starting cells from scratch.

Contents

1 Overview **2**

2 The model **3**

3 Certifying one probe set: the rewrite engine **4**

4 From one probe set to a safe set: the extension **4**

5 Covering the probe space: the cell search **5**
5.1 Cells . . . 5
5.2 Splitting a cell along a safe set . . . 5
5.3 Remembering safe sets . . . 6
5.4 Visiting a cell . . . 6

6 Symmetry **7**
6.1 What a symmetry renames: atoms . . . 7
6.2 What the group acts on . . . 8
6.3 Chain contraction . . . 8
6.4 Why atoms and not raw leaves . . . 10
6.5 From generators to wire tables . . . 11
6.6 Groups, orbits and stabilisers in this setting . . . 11

7 Symmetry use 1: dominance before the engine **11**

8 Symmetry use 2: orbital branching for the starting cells **12**
8.1 The goal, and why not canonical forms . . . 12
8.2 One wire . . . 12
8.3 The branching rule . . . 12
8.4 Repeating it over all orbits: the shape of the first level . . . 12
8.5 The remaining wires: stabilisers and recursion . . . 13
8.6 Partial symmetry: stable domain and loose wires . . . 13
8.7 Branching only where it pays . . . 14
8.8 Correctness . . . 14

9 The complete algorithm **15**

10 Worked example: DOM-AND with 3 shares at order 2 **15**

11 How the implementation is checked **16**

12 Measurements **17**

13 Limitations and open questions **17**

1 Overview

The question is whether a circuit is *d-probing secure*: whether every set of at most d wires, observed together, reveals nothing about the secrets. There are $\binom{n}{d}$ such sets for a circuit with n wires — 1 873 200 for the 4-share DOM Keccak χ at $d = 3$ — so no tool can afford to examine them one by one. The verifier combines four ideas, each of which removes a different kind of redundancy:

(1) Certify one probe set with a rewrite engine based on optimistic sampling (Section 3). Besides the verdict, the engine returns the change of random variables that proved it — the coupling.

(2) Extend the certified probe set: every wire that the same coupling makes secret-free can be observed together with it. This gives a *safe set*, often of dozens of wires, and every subset of it is secure (Section 4).

(3) Search the probe space in cells: blocks of probe sets described by a few numbers, split along safe sets, with every safe set remembered so that the engine is called only on probe sets nothing stored covers (Section 5).

(4) Use the symmetry of the circuit: an automorphism maps safe sets to safe sets. It is used to start from cells that contain at least one representative of every orbit of probe sets (Section 8) and to recognise probe sets that are symmetric images of covered ones (Section 7).

Where things live.

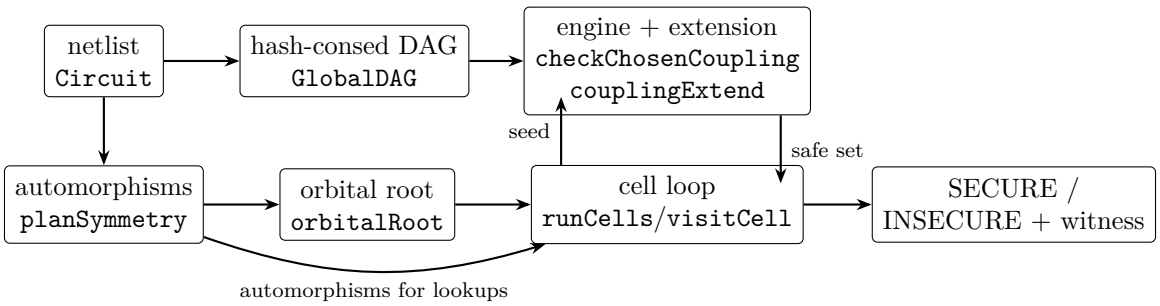


Figure 1: The pipeline. The entry point is `checkDProbingSym` (`Symmetry.lean`); the symmetry-free variant is `checkDProbingCells` (`CellSearch.lean`); the older split search is `checkDProbing` (`ProbeClosure.lean`).

file	contents
`Verif/n-aryDAG.lean`	hash-consed $\mathbb{F}_2$ DAG; per-node facts (secret dependence, random support, PIT values)
`Verif/Circuit.lean`	netlist (`Circuit`), bridge to the DAG (`GlobalDAG`)
`Verif/Engine.lean`	rewrite engine: `rewriteComplete`, `checkProbeCompleteT`, `checkProbeRoots`
`Verif/Coupling.lean`	extension: `couplingExtend`, `coneEval`, ANF fallback
`Verif/ProbeClosure.lean`	split search (`checkDProbing`), `checkChosenCoupling`, `Stats`
`Verif/CellSearch.lean`	cells, `childCells`, `SafeStore`, `visitCell`, `runCells`, audits
`Verif/Automorphism.lean`	automorphism groups by individualisation–refinement, `contractAssoc`
`Verif/Symmetry.lean`	group → wire tables, orbital root, `planSymmetry`, `checkDProbingSym`
`Main.lean`	gadget suite, `check`, `trace`, `golden`, `bench`

A complete run on one gadget, step by step, is printed by `lake exe verif trace "DOM AND (3 shares)"`; Section 10 walks through it.

2 The model

Circuits. A circuit has *leaves* of three kinds — secrets s, randoms r (independent and uniform over $\mathbb{F}_2$) and publics — *input shares*, and *body wires*. Every input share and every wire is defined by a gate, an XOR or an AND of earlier wires, shares, leaves or constants. Input shares encode the secrets: the standard sharing of a secret a into n shares uses $n-1$ fresh randoms, $a_i = r_{a,i}$ for $i < n-1$ and $a_{n-1} = a \oplus \bigoplus_i r_{a,i}$. The body wires are what an attacker may probe; input shares are not probe targets (`Circuit.inputOrder` versus `Circuit.wireOrder`). Gates are binary, because in the probing model every gate output is an observable: an n-ary sum hides $n-2$ intermediate values.

Each wire w computes a function $[\![w]\!]$ of the leaves. For a set P of wires, $[\![P]\!]$ is the tuple of their values, and for fixed secrets and publics its *distribution* is taken over the uniform randoms.

Definition 2.1 (security of a probe set; safe set)**.** A set P of wires is *secure*, $\mathrm{Sec}(P)$, if the distribution of $[\![P]\!]$ is the same for all values of the secrets (for every value of the publics). We call a set of wires that is secure a *safe set*. The circuit is *d-probing secure* if every set of at most d body wires is secure.

Lemma 2.2 (subsets)**.** *Every subset of a safe set is safe.*

Proof. The distribution of a sub-tuple is a marginal of the distribution of the tuple; if the latter does not depend on the secrets, neither does the former. □

Assuming $n \geq d$, it therefore suffices to check every set of *exactly* d wires: smaller sets are subsets of these. The converse direction fails, and this matters later.

Remark 2.3 (unions are not safe). Two safe sets need not have a safe union. With a secret s and a random r, the wire r alone and the wire $s \oplus r$ alone are each uniform, hence safe, but together they reveal s. So the search may never conclude that a probe set is secure because it lies in the union of two safe sets; it must lie inside *one*.

The DAG. The engine works on a hash-consed DAG of n-ary, canonically ordered XOR and AND nodes (`n-aryDAG.lean`): building a node flattens nested XORs, cancels repeated XOR children, and returns the existing node if the same expression was built before. Equal expressions are therefore equal node ids. Each node also stores, computed once from its children when it is created (`DAG.alloc` (`n-aryDAG.lean`)): whether it depends on a secret (`secretDep`), the set of randoms it depends on, as a bitset (`rndSupp`), and its values under 64 parallel pseudo-random evaluations (`valR`, `valZ`; used by the extension's fast test).

3 Certifying one probe set: the rewrite engine

The engine (`Engine.lean`) decides $\mathrm{Sec}(P)$ for a probe set P syntactically, by *optimistic sampling*.

Lemma 3.1 (optimistic sampling)**.** *If a random r does not occur in an expression e, then the map $r \mapsto e \oplus r$ (all other leaves fixed) is a bijection of the randoms that preserves the uniform distribution. Hence substituting $e \oplus r$ for r in every observed expression leaves the joint distribution of the observations unchanged.*

The engine applies such substitutions until no observed expression mentions a secret; then P is secure. It uses two rules (`rewriteComplete` (`Engine.lean`)):

- **Simple rule.** A random r that occurs exactly once, inside an XOR node $x = e \oplus r$, and is not itself observed: after the substitution $r \leftarrow x$, the node x becomes the bare random r — the whole subexpression e disappears. (`findSimpleRandom`.)
- **General rule.** A random r with an additive occurrence $x = e \oplus r$, $r \notin e$, but other occurrences too: substitute $r \leftarrow x$ everywhere; the matched occurrence collapses to r and the others absorb e, cancelling where they share it. (`findGeneralRandom`.)

When both rules stall, the engine *factors* ($a \cdot b \oplus a \cdot c \to a \cdot (b \oplus c)$, `DAG.factor` (`n-aryDAG.lean`)) to expose randoms buried in products, and retries. This is what certifies, for example, the quadratic S-box Q_{12}^4, which a witness replay without factoring rejects. The engine is sound but incomplete: failure means "could not prove it", and the search reports the failed probe set as a counterexample.

The coupling. The sequence of substitutions $T = (r_1 \leftarrow x_1), \ldots, (r_k \leftarrow x_k)$ is the *coupling* that certified P. It is returned by `checkProbeCompleteT` (`Engine.lean`) and consumed by the extension. (A faster, reference-counted variant of the simple rule, `checkProbeRoots`, is used only by the split search for verdicts without a coupling.)

4 From one probe set to a safe set: the extension

Theorem 4.1 (coupling extension)**.** *Let T be a coupling under which every expression of the probe set P is secret-free. Let S be the set of all wires w such that $w \circ T$ — the expression of w after the substitutions of T — is secret-free. Then $S \supseteq P$ and S is a safe set.*

Proof. T is a composition of distribution-preserving bijections of the randoms, so the joint distribution of $[\![S]\!]$ equals that of the tuple of expressions $(w \circ T)_{w \in S}$. Each of them is a function of the randoms (and publics) only, so the tuple is too, and its distribution does not depend on the secrets. □

The point is that *one* coupling serves all wires of S at once, which is why they are safe *jointly*, not only one by one. The implementation (`couplingExtend` (`Coupling.lean`)) decides membership exactly, but avoids most of the work:

- A wire whose random support contains none of the substituted randoms is *untouched*: $w \circ T = w$, and its stored secret-dependence bit decides.
- For a touched wire, a fast rejection test evaluates $w \circ T$ on 64 random assignments twice — secrets random, secrets zero — re-evaluating only the part of the DAG that depends on the substituted randoms (`coneEval`). Any difference proves secret dependence.
- Survivors are accepted only by an exact test: the substituted expression is syntactically secret-free, or its algebraic normal form is (`anfSecretFreeM`).

The chosen probe set is always included, which is what guarantees progress in the search. Since the 2026-10-28 changes an extension costs 20–40 μs on the large gadgets (the `engine-bench` files in `lean_ffi/verif/results/`).

5 Covering the probe space: the cell search

5.1 Cells

Definition 5.1 (cell)**.** A *cell* is a list of pairs $\langle c_1 : P_1 \rangle \times \cdots \times \langle c_m : P_m \rangle$ of pairwise disjoint wire sets P_i and counts c_i. It stands for the set of probe sets obtained by choosing c_i wires from each P_i; it contains $\prod_i \binom{|P_i|}{c_i}$ probe sets (`probeCount` (`CellSearch.lean`)). Its *seed* is the probe set made of the first c_i wires of each P_i.

The *root cell* $\langle d : W \rangle$ contains every probe set of size d. A cell is a compact description of a large, structured block of probe sets: the root cell of Keccak χ with 4 shares is one pair of numbers describing 1 873 200 probe sets.

5.2 Splitting a cell along a safe set

Lemma 5.2 (split)**.** *Let $C = \prod_i \langle c_i : P_i \rangle$ be a cell and S a set of wires. Write $P_i^{\text{in}} = P_i \cap S$ and $P_i^{\text{out}} = P_i \setminus S$. Then C is the disjoint union, over all choices $0 \le j_i \le c_i$, of the cells*

$$\prod_i \langle j_i : P_i^{\text{in}} \rangle \times \langle c_i - j_i : P_i^{\text{out}} \rangle.$$

The member with $j_i = c_i$ for all i consists of probe sets contained in S.

Proof. A probe set of C picks c_i wires from each P_i; let j_i be how many of them lie in S. This determines the member uniquely. For a single part this is Vandermonde's identity $\binom{|P|}{c} = \sum_j \binom{|P \cap S|}{j} \binom{|P \setminus S|}{c-j}$. □

If S is safe, the all-in member is settled by Lemma 2.2 and is dropped; the other members are the *child cells* (`childCells` (`CellSearch.lean`)).

5.3 Remembering safe sets

Every safe set the search obtains is stored (`SafeStore`). For every wire the store keeps a bitset over the stored sets that contain it, so "which stored sets contain all of the wires O?" is the AND of $|O|$ bitsets (`SafeStore.find` (`CellSearch.lean`)). Each stored set is also kept as a wire bitset, for choosing which one to split along. Two design choices distinguish this search from maskVerif and from the split search of `ProbeClosure.lean`. Both were measured against their opposites (`lake exe verif bench ablate`, `results/search-ablation_2026-10-30.txt`):

- Every certified probe set is extended over *all* wires of the circuit, not only the wires of the current cell, and the result is kept for the rest of the run. maskVerif extends over the current cell and keeps nothing. Limiting the extension to the cell raises the suite from 989 to 23 316 engine calls and from 161 to 1 342 ms; only on TI Fides AB1 at $d = 1$ is it faster, 17 against 24 ms, with the same number of engine calls.
- There is no *union attempt* (asking the engine to certify all wires of a cell at once). Union attempts are not useless: they often succeed (93 % of the attempts on the suite when made just before an engine call) and cut engine calls by up to a factor of 7. They cost more than they save, though. With our full engine the suite goes from 161 to 1 732 ms. With the simple rule only, the form maskVerif uses at every cell, it goes to 211–235 ms.

5.4 Visiting a cell

Algorithm 1 One cell visit (`visitCell` (`CellSearch.lean`))

Require: cell C, store $\mathcal{S}$ of safe sets, automorphisms H (possibly empty)

if some $S \in \mathcal{S}$ contains every wire of C **then return** no children ▷ skip
$p \leftarrow$ seed of C
if some $S \in \mathcal{S}$ contains p **then**
    return split(C, S^*) ▷ cover; S^*: the hit covering most of C
if some $h \in H$ and $S \in \mathcal{S}$ have $h^{-1}(p) \subseteq S$ **then** ▷ Section 7
    add $h(S)$ to $\mathcal{S}$; **return** split$(C, h(S))$
run the engine on p ▷ `checkChosenCoupling`
if not certified **then**
    stop: INSECURE, counterexample p
$S \leftarrow$ extension of p over all wires ▷ `couplingExtend`
add S to $\mathcal{S}$; **return** split(C, S)

The search keeps a stack of cells, starting from the root cell(s), and visits them depth-first until the stack is empty (`runCells` (`CellSearch.lean`)).

Theorem 5.3 (cell search)**.** *Started from cells that together contain every probe set of size d, the search terminates. If it reports SECURE, the circuit is d-probing secure; if it reports INSECURE, the reported probe set could not be certified by the engine.*

Proof. Invariant: every probe set of the starting cells lies in a cell on the stack or inside a stored safe set. A skip removes a cell inside a stored set; a split replaces a cell by its children and a set that is stored (Lemma 5.2). Stored sets are safe: they come from Theorem 4.1 or are automorphic images of safe sets (Theorem 6.2). When the stack is empty every starting probe set is inside a safe set, hence secure (Lemma 2.2); Theorem 8.4 extends this to all probe sets when the starting cells are the orbital root. *Termination:* the seed always lies in the set the cell is split along (the engine's extension contains the seed; a cover contains it by construction), so

the dropped all-in member is non-empty and every child has strictly fewer probe sets than its parent. □

> Why cells and not probe sets? A safe set S of 30 wires settles $\binom{30}{3} = 4060$ probe sets of size 3, but they are scattered through any list of probe sets. In the cell picture they are exactly one member of a split, removed in one operation; the other members are described by a handful of numbers each. The search never lists the probe sets it settles.

Without symmetry ($H = \emptyset$, root $\langle d : W \rangle$) this is `checkDProbingCells` (`CellSearch.lean`). On the 41-entry suite it is 26 times faster than the split search (126 ms against 3 271 ms), but Keccak χ with 4 shares still needs 14 706 engine calls and nearly two seconds. Symmetry removes most of these.

6 Symmetry

6.1 What a symmetry renames: atoms

A symmetry should say "share 0 and share 1 play the same role". So what gets renamed has to be the shares themselves, not the leaves that their formulas happen to mention. Take a 2-share sharing $a_0 = r_a$, $a_1 = a \oplus r_a$. Exchanging the two shares cannot be written as a renaming of leaves: it would need $r_a \mapsto a \oplus r_a$. Yet (a_0, a_1) and (a_1, a_0) have the same distribution: both are a uniform pair with XOR a. The symmetry is present in the distribution and absent from the formula, because one share must be written down as the *carrier* of the secret, although any other share could have played that role.

So we stop looking inside the sharing. The *atoms* of a circuit are its input shares, its *gadget randoms* (the randoms read by body wires) and its publics. Every body wire is an expression over atoms (`buildAtomicDAG` (`Automorphism.lean`)). This hides the formula that names the carrier, and it needs two conditions, both checked on the original circuit (`shareChecks` (`Automorphism.lean`)):

(S1) Each sharing of size n is a uniform XOR sharing: $n - 1$ bare fresh randoms and one carrier $x \oplus \bigoplus r$ of exactly those randoms; sharings use disjoint randoms (`sharingsOf`, `checkExchangeable`).

(S2) No body wire reads a secret or an encoding random directly, so the body sees the encoding only through the share values (`checkPrivateEncoding`).

A circuit that fails them gets no symmetry: its group is trivial, and the tool runs the cell search of Section 5 from the single root cell.

Definition 6.1 (relabelling; automorphism)**.** A *relabelling* σ is a bijection of the atoms that maps *whole sharings onto whole sharings*, gadget randoms to gadget randoms and publics to publics. It acts on expressions by renaming atoms. It *maps wire w to wire w'* if relabelling the expression of w gives exactly the expression of w'; where no wire computes the relabelled expression, σ is *not valid* on w. An *automorphism* is such a σ together with its (partial, injective) action on the wires.

Theorem 6.2 (transport; memo Theorem 1)**.** *Assume (S1) and (S2). If σ is valid on every wire of P, then* $\mathrm{Sec}(P) \iff \mathrm{Sec}(\sigma(P))$. *In particular the image of a safe set under an automorphism is a safe set (restricted to the wires on which it is valid).*

Proof. By (S2) each body wire w is a function f_w of the share vector x, the gadget randoms r and the publics. Validity gives $f_{\sigma(w)}(x, r) = f_w(x \circ \sigma, r \circ \sigma)$ on P. Fix the secrets s. In $x \circ \sigma$, the block of sharing k is a reordering of the block of sharing $\pi(k)$, where π is the permutation of sharings induced by σ. By (S1) that block is a uniform sharing of $s_{\pi(k)}$ whatever its order, and

sharings are independent; the gadget randoms are permuted, which preserves their distribution, and permuting the publics only reindexes the quantifier over them. So the distribution of $\sigma(P)$ under secrets s equals that of P under secrets $s \circ \pi$. If the latter does not depend on the secrets, neither does the former; the converse uses σ^{-1}. □

Remark 6.3 (one direction only). The engine is sound but not equivariant: it may certify P and fail on $\sigma(P)$ (its tie-breaking depends on node ids). A *failure* is therefore never transported; only safety is.

Remark 6.4 (whole sharings are a real hypothesis). An arbitrary bijection of atoms is *not* safe. With 2-share sharings of a and b, the wires $v = a_0 \oplus a_1$ and $w = a_0 \oplus b_1$, and the swap $a_1 \leftrightarrow b_1$, the wire map $v \mapsto w$ is perfect, yet v equals the secret a and w is uniform.

6.2 What the group acts on

Two facts make the rest of this section fit together.

The group lives on the atoms. A group element renames atoms. Its action on wires is derived from that, wire by wire, and may be undefined on some wires. Composing, inverting and listing elements all happen on the atoms, where every element is a true permutation.
Transport looks only at the probe set. Theorem 6.2 asks that σ be valid on the wires of P. It says nothing about the other wires of the circuit, so σ does not have to be a symmetry of the whole circuit to be useful.

The classical notion, an automorphism of the netlist graph, is much stronger than we need: it must give an image to *every* wire, so a single wire without an image rejects the whole relabelling. What we want instead is relabellings that are valid on *many* wires.

6.3 Chain contraction

The carrier was one accident of notation. The bracketing of sums is the other. A gate has two inputs, so the sum $u_2 \oplus c_{20} \oplus c_{21}$ must be bracketed: the tool writes $s_2 m_0 = u_2 \oplus c_{20}$ and $s_2 = s_2 m_0 \oplus c_{21}$, though the other bracketing writes the same sum. (The bracketing decides which intermediate wires exist, so it can affect security; see the remark below.) *Contraction* undoes this choice during the search for symmetries. The table gives the order of the group on the netlist as built and on the contracted netlist.

gadget	as built	contracted
DOM-AND, 2 shares	2	4
DOM-AND, 3 shares	1	36
DOM-AND, 4 shares	1	576
DOM-AND, 5 shares	1	14 400
DOM Keccak χ, 3 shares	5	10
DOM Keccak χ, 4 shares	5	30
TI-AND, 3 shares	3	6
ISW-AND, 5 shares	1	2

The problem. Take DOM-AND with 3 shares and the relabelling τ that swaps share indices 0 and 1 ($a_0 \leftrightarrow a_1$, $b_0 \leftrightarrow b_1$, $r_{02} \leftrightarrow r_{12}$). It sends $s_0 \mapsto s_1$, $c_{01} \mapsto c_{10}$, and so on, and it fixes s_2, since it merely exchanges c_{20} and c_{21} inside the sum. But $s_2 m_0 = u_2 \oplus c_{20}$ would become $u_2 \oplus c_{21}$, and no wire computes that. So τ is valid on 20 of the 21 wires, and a search for exact netlist automorphisms still rejects it because of the one remaining wire. The same happens to every share permutation: the as-built group is trivial.

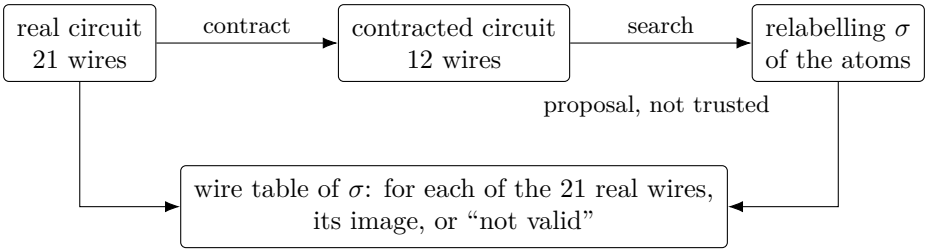
Note that the wire s_2m_0 really has no partner. This is not a defect of the search: probe sets containing s_2m_0 genuinely cannot be mapped by τ. The other 20 wires should not be made to suffer for it.

The idea. Searching directly for the relabelling with the most valid wires is a hard, largest-common-subgraph style problem. Contraction is a cheap substitute: hide the wires most likely to be in the way, search on what is left, and afterwards ask each hidden wire whether it has an image.

Definition 6.5 (contraction, `contractAssoc` (`Automorphism.lean`))**.** A body wire is *absorbed* if it is used by exactly one gate and that gate has the same operator; its inputs move into that gate, which becomes n-ary. This is repeated until no such wire remains. The other wires are the *surviving* wires.

In DOM-AND with 3 shares the 6 wires c_{ij} and the 3 wires s_im_0 are absorbed, and 12 wires survive: u_i, p_{ij} and s_i, with for instance $s_0 = u_0 \oplus p_{01} \oplus r_{01} \oplus p_{02} \oplus r_{02}$. The contracted circuit is the same circuit with some inner points of its large sums left unnamed: *it has exactly the same atoms*, and every surviving wire computes the same function. (Only XOR-into-XOR and AND-into-AND are absorbed, since those operators are associative and commutative.)

The pipeline.



1. **Propose.** Search for automorphisms of the contracted netlist. Each one is a pair: a relabelling σ and a map ϕ on the surviving wires.
2. **Keep only σ.** The map ϕ is thrown away (`atomicLeafMap`). Nothing has to be "brought back" to the original circuit: σ renames atoms, and the original circuit has the same atoms.
3. **Dispose.** For every real wire w, relabel the expression of w and look the result up among the real wires (`nameMap` (`Symmetry.lean`)). A hit gives the image of w; no hit means σ is not valid on w (`WireTables.of` (`Symmetry.lean`)).

For τ this returns images for all 12 surviving wires and for 8 of the 9 absorbed ones, for example $c_{02} \mapsto c_{12}$ and $s_0m_0 \mapsto s_1m_0$, and "not valid" on s_2m_0. For the relabelling that swaps only $b_0 \leftrightarrow b_1$, it returns images for 16 wires: $u_0 \mapsto p_{01}$, $p_{20} \mapsto p_{21}$, and $s_0 \mapsto s_0$ (the sum $a_0 \cdot (b_0 \oplus b_1 \oplus b_2) \oplus r_{01} \oplus r_{02}$ does not change), and "not valid" on $c_{01}, c_{10}, c_{20}, c_{21}, s_2m_0$.

Soundness. Contraction is not part of the soundness argument at all:

Theorem 6.6 (the proposal is not trusted)**.** *Let σ be* any *relabelling in the sense of Definition 6.1, wherever it came from, and let h_σ be its wire table on the real circuit. Then Theorem 6.2 holds for h_σ on every probe set in its valid part.*

Proof. Theorem 6.2 has two hypotheses: σ is a relabelling, and for each $w \in P$ the relabelled expression of w is the expression of $h_\sigma(w)$. The second is what `nameMap` tests, on the real circuit's expressions. Neither hypothesis mentions a contracted circuit, a graph or a search. □

The one thing the search has to deliver is therefore that σ is a relabelling, which Remark 6.4 shows is not automatic. It is enforced exactly: the search graph has one extra vertex per sharing, joined to its shares (`buildShareGraph`), and `verifyMap` accepts a candidate only if these edges are preserved. Contraction does not touch input shares, so these vertices are the same in the contracted graph, and composing relabellings gives a relabelling.

If contraction were wrong, or replaced by a random guess of a relabelling, the only effect would be tables with few valid entries, hence less pruning. The contracted circuit is a heuristic that decides *which* relabellings get tried; it never decides whether one is correct. Contraction also gives two guarantees of quality:

Remark 6.7 (absorbed wires are still probed). Contraction affects only the symmetry proposal. The search runs over all real wires, so an absorbed wire remains a probe position, covered by the engine or by a table that is valid on it. This is essential, because contraction does not preserve security. *Trichina AND* and *Trichina AND (bad assoc)* differ only in the bracketing of one sum, and they contract to the *identical* circuit (`bench trichina`), with $c_1 = a_0b_0 \oplus a_0b_1 \oplus a_1b_0 \oplus a_1b_1 \oplus r$. The first is secure. The second is insecure, because its absorbed wire $t_3 = a_0b_0 \oplus a_0b_1 \oplus a_1b_0 \oplus a_1b_1$ equals ab unmasked. The tool rejects it, because t_3 was never dropped from the search.

Lemma 6.8 (contraction loses nothing)**.** *(a) Every automorphism of the as-built netlist induces one of the contracted netlist with the same relabelling. So contraction can only enlarge the group.*

(b) If (σ, ϕ) is an automorphism of the contracted netlist, then σ sends the expression of each surviving wire w to the expression of $\phi(w)$. So only absorbed wires can lose their image.

Proof. (a) "Used once, by a gate with the same operator" is a property that automorphisms preserve, so an automorphism permutes the absorbed wires. It therefore maps the flattened inputs of a surviving gate to the flattened inputs of its image. (b) By induction along the contracted gates: the expression of w is its operator applied to the expressions of its inputs, and ϕ preserves both. □

Both hold on the whole suite: every as-built order divides the contracted order, and under every listed element, every surviving wire goes to an expression that some wire computes. The tables are stricter than this in one respect: they also refuse a wire whose expression, or whose image's expression, is shared with another wire, because a relabelling is not a permutation there (`dup` in `WireTables.of`).

6.4 Why atoms and not raw leaves

One could avoid (S1) and (S2) by searching for renamings of the raw leaves of the formulas. Such a renaming needs no assumption on the encoding, but it can never move a carrier, and that is all it gives up and all that separates it from the group above:

Lemma 6.9 (raw leaves keep the carriers)**.** *Assume (S1) and (S2). The automorphisms that rename raw leaves are exactly the elements of the group of relabellings that send carriers to carriers. In particular, their number is the order of the group divided by the size of the orbit of the set of carriers.*

Proof. An automorphism on raw leaves preserves input shares, and it preserves "two shares have a common encoding random", hence sharings and carriers; by (S2) encoding randoms are exactly the randoms read by input shares, so gadget randoms go to gadget randoms. So it restricts to a relabelling that keeps the carriers. Conversely, a relabelling keeping the carriers lifts to the leaves: send the random of a bare share to the random of its image, and each secret to the secret of the image sharing. The carrier formula is preserved, because it is the secret plus all randoms of its sharing. □

In DOM-AND with 3 shares the group is $S_3 \times S_3$, of order 36; keeping a_2 and b_2 in place leaves $2 \times 2 = 4$. Likewise $576/16 = 36$ with 4 shares and $14\,400/25 = 576$ with 5. So renaming raw leaves adds nothing where (S1) and (S2) hold. The comparison below shows what it loses (`lake exe verif bench contraction`, time in ms and engine calls, setup included; the last row was measured on the version of the code that still had the raw-leaf search, `results/group-ablation_2026-10-31.txt`):

	41-entry suite		Keccak χ, 4 shares		DOM-AND, 5 shares	
symmetry used	ms	engine	ms	engine	ms	engine
atoms, contracted (the tool)	127	993	128	579	65	239
atoms, as built	122	1 278	552	2 989	108	843
raw leaves, contracted	132	1 199	389	2 303	99	435

Verdicts are identical in all rows. Differences of a few ms on the suite are noise: it is dominated by small gadgets, where symmetry hardly changes the running time. On Keccak χ with 4 shares, atoms take a third of the time of raw leaves, and contraction takes a quarter of the time of the as-built netlist, where the group shrinks from 30 to 5.

6.5 From generators to wire tables

The generators are closed into a list of group elements (`closeGroup`). Closure happens on the atoms, so no partial wire maps are ever composed; each listed element gets its own wire table by Theorem 6.6's exact test. A group larger than a bound B (default 64, `defaultGroupCap`) is replaced by the largest closed subgroup of order at most B found by adding generators one at a time (`closeIncremental`). Any subgroup is sound; a smaller one costs less to set up and to look up through, and prunes less (Section 12).

Each element thus becomes a *wire table*: an array giving, for each wire, its image, or "not valid". Tables are injective where valid, so inverses exist (`invertTable` (`Symmetry.lean`)).

6.6 Groups, orbits and stabilisers in this setting

The group G acts on wires and, element by element, on probe sets: $h(P) = \{h(w) : w \in P\}$ when h is valid on every wire of P. The *orbit* of a probe set P is the set of all its images. By Theorem 6.2, all members of an orbit are secure or none is: **it suffices to establish security for one representative of every orbit.** The *stabiliser* of a wire x is the subgroup $\mathrm{Stab}(x) = \{h \in G : h(x) = x\}$.

Two uses of this fact follow. The first is local and simple (Section 7); the second shapes the whole search (Section 8).

7 Symmetry use 1: dominance before the engine

When no stored safe set contains a cell's seed p, the engine would be called. Before that, the search asks whether a symmetric image of a stored set contains p:

Lemma 7.1 (dominance)**.** *If h is an automorphism, S a stored safe set, and $h^{-1}(p) \subseteq S$ (that is, h^{-1} is valid on p and maps it into S), then $p \subseteq h(S)$ and $h(S)$ is safe.*

Proof. Every wire of p is h of a wire of S, so lies in $h(S)$; $h(S)$ is safe by Theorem 6.2. □

The search maps p backwards — d table lookups per automorphism — and asks the store as before (`findThroughSymmetry` (`CellSearch.lean`)). The image $h(S)$ is then stored as an ordinary safe set and the cell is split along it, so later lookups find it without symmetry. This is the probing-security analogue of symmetry breaking by dominance detection in constraint

programming [6, 7]: a subproblem is skipped if a symmetric image of it is dominated by work already done.

Two details matter for speed. The lookup through symmetry runs only when the alternative is an engine call, not at every cell. And mapping the *seed* backwards (instead of storing every image of every safe set) keeps the store's size independent of the group order.

8 Symmetry use 2: orbital branching for the starting cells

Dominance removes engine calls, but the search still *visits* cells for probe sets whose symmetric images were already handled. On Keccak χ with 4 shares, searching from the single root cell with dominance lookups visits 61 572 cells and takes 414 ms; starting from the orbital root it visits 13 415 cells and takes 130 ms. This section explains what the orbital root is and why it has its shape.

8.1 The goal, and why not canonical forms

We want starting cells that contain *at least one* member of every orbit of probe sets — ideally few more. A textbook approach lists all probe sets and keeps those that are lexicographically smallest in their orbit (a canonical-form test, as in orderly generation [8]). That defeats the purpose here: it lists every probe set, while the cell search is built to avoid listing any. We need cells — large blocks described by a few numbers — that are guaranteed to contain a representative of every orbit. Orbital branching produces exactly that.

8.2 One wire

For $d = 1$ the answer is immediate: a probe set is one wire, its orbit is the wire's orbit, and one wire per orbit suffices.

8.3 The branching rule

Now take d arbitrary and one orbit O of wires, with a chosen representative $x \in O$. Every probe set either meets O or avoids it.

Lemma 8.1 (orbital branching)**.** *If a probe set P meets O, some $h \in G$ maps P to a probe set containing x.*

Proof. Pick $y \in P \cap O$; since y and x are in the same orbit, some h has $h(y) = x$, so $x \in h(P)$. □

So the probe sets that meet O need only be examined *in the form that contains x*:

$$\{P : P \cap O \neq \emptyset\} \;\rightsquigarrow\; \{P : x \in P\}, \qquad \{P : P \cap O = \emptyset\} \text{ kept as is.}$$

This is the two-way branching of *orbital branching* in integer programming [4]: on an orbit O of 0/1 variables, branch into "$x_j = 1$ for one representative $j \in O$" and "$\sum_{i \in O} x_i = 0$"; the other $|O| - 1$ branches "$x_i = 1$" are symmetric copies of the first and are never explored. Here the variables are "wire w is probed" and $\sum_w x_w = d$.

8.4 Repeating it over all orbits: the shape of the first level

Order the orbits of wires $O_1, O_2, \ldots, O_k$ and fix a representative $x_i \in O_i$. Apply the rule to O_1: probe sets that meet O_1 become "contains x_1"; the rest avoid O_1. Apply it again, to O_2, on the probe sets that avoid O_1, and so on. After the last orbit nothing is left (a probe set meets some orbit). The pieces are:

$$\boxed{\langle 1 : \{x_i\}\rangle \;\times\; \langle d-1 : (O_i \setminus \{x_i\}) \cup O_{i+1} \cup \cdots \cup O_k\rangle \qquad i = 1, \ldots, k.}$$

This is the shape of the starting cells: **the i-th cell is pinned to the representative of orbit i, and the other $d-1$ wires come from the rest of that orbit and the later orbits.** The earlier orbits are missing because a probe set reaching cell i was, by construction, one that avoids $O_1, \ldots, O_{i-1}$ — each probe set is assigned to the *first* orbit it meets. (Missing them is not needed for soundness; it is what makes the cells disjoint, so nothing is examined twice for that reason.)

Example 8.2 (two orbits, a rotation)**.** Six wires $a_0, a_1, a_2, b_0, b_1, b_2$ and the rotation $\rho : a_i \mapsto a_{i+1},\ b_i \mapsto b_{i+1}$ (indices mod 3), generating a group of order 3. For $d = 2$ there are 15 pairs; no pair is fixed by ρ, so there are $15/3 = 5$ orbits. The orbits of wires are A and B. The branching gives

$$\langle 1 : \{a_0\}\rangle \times \langle 1 : \{a_1, a_2, b_0, b_1, b_2\}\rangle \text{ (5 pairs)}, \qquad \langle 1 : \{b_0\}\rangle \times \langle 1 : \{b_1, b_2\}\rangle \text{ (2 pairs)},$$

7 pairs instead of 15. Two orbits are still represented twice: $\{a_0, a_1\}$ and $\{a_0, a_2\}$ (ρ^2 maps the first to the second), and likewise $\{b_0, b_1\}$, $\{b_0, b_2\}$. The reason is that a pair with *two* wires in its first orbit can be rotated to contain x in two ways. These remaining duplicates are what dominance (Section 7) removes when the search reaches them.

8.5 The remaining wires: stabilisers and recursion

Inside cell i, the wire x_i is fixed. Every element that fixes x_i — the stabiliser $\mathrm{Stab}(x_i)$ — maps the cell's probe sets to probe sets that still contain x_i, so it is still a symmetry of the sub-problem "choose the remaining $d-1$ wires from $F_i = (O_i \setminus \{x_i\}) \cup O_{i+1} \cup \ldots$". So we branch again, inside F_i, on the orbits of $\mathrm{Stab}(x_i)$, with the same rule. Each level pins one more wire and passes to a smaller group, until all d wires are pinned or the group becomes trivial; then the remaining choice is left as a plain block. This is a stabiliser chain, and the cells are the leaves of the branching tree (`obranch` (`Symmetry.lean`), `pinClass` (`Symmetry.lean`)).

Example 8.3 (a ring, with and without reflections)**.** Six wires $w_0, \ldots, w_5$ on a ring, $d = 2$: 15 pairs in 3 orbits (distance 1, 2, 3).

- Under the rotations C_6 there is one orbit of wires; the first level gives $\langle 1 : \{w_0\}\rangle \times \langle 1 : \{w_1, \ldots, w_5\}\rangle$ (5 pairs). The stabiliser of w_0 in C_6 is trivial, so the recursion stops: 5 pairs for 3 orbits ($\{w_0, w_1\}$ and $\{w_0, w_5\}$ are the same orbit).
- Under the dihedral group D_6 (rotations and reflections) the stabiliser of w_0 contains the reflection $w_i \leftrightarrow w_{-i}$. Its orbits on the remaining wires are $\{w_1, w_5\}$, $\{w_2, w_4\}$, $\{w_3\}$, and branching on them gives the three cells $\{w_0, w_1\}$, $\{w_0, w_2\}$, $\{w_0, w_3\}$ — exactly one pair per orbit.

The larger the stabilisers, the closer the cells come to one representative per orbit.

Orbits of size one (wires every element fixes) gain nothing from pinning; they are kept together as one plain part ("`fix`" in `pinClass`).

8.6 Partial symmetry: stable domain and loose wires

So far every element was valid on every wire. In real circuits it is not: a chain intermediate like $s_0 m_0 = u_0 \oplus c_{01}$ has no image under most share permutations. Lemma 8.1 needs, for a probe set P, an element valid on *all* of P; and the recursion composes elements, so validity must survive composition. The construction therefore works on the *stable domain* of the current group K inside the current wire set F:

$$D = \{w \in F : \text{every } h \in K \text{ is valid on } w \text{ and } h(w) \in F\} \qquad (\texttt{stableIn}).$$

Because K is a closed group and validity is decided exactly, D is mapped onto itself by K, every composition of elements stays valid on D, and the branching of the previous subsections is applied to D. This is why the group is *listed and closed* rather than kept as generators: orbits and stabilisers on D are then those of a genuine group acting by permutations.

The wires of $F \setminus D$ are *loose*. Probe sets that touch a loose wire are handled in three steps (`looseClass` (`Symmetry.lean`)):

1. look for a subgroup $K_2 \leq K$ — the elements mapping some set of loose wires onto itself — whose stable domain is larger and covers some loose wires (`bestSubgroup`), and branch with K_2 on the newly covered wires;
2. a loose wire whose own stabiliser still moves other wires gets its own pinned cell, with that stabiliser for the recursion;
3. the remaining loose wires form plain cells: j of them, and $d - j$ wires of D.

In DOM-AND with 3 shares the products u_i, p_{ij} and the outputs s_i form the stable domain of the group; the masked terms c_{ij} are loose but have a subgroup that permutes them; the chain intermediates $s_i m_0$ are loose with small stabilisers. The resulting root is listed in Section 10.

8.7 Branching only where it pays

Cutting a cell by orbits replaces it by several smaller cells. If the group is small, the cut removes few probe sets but fragments the cell, and large cells serve the stored safe sets better (one safe set removes a whole member of a large cell in one split). So each branching step is kept only if it removes at least a quarter of the cell's probe sets; otherwise the cell is kept whole (in `obranch` (`Symmetry.lean`)). For ISW-AND with 5 shares (a group of order 2) the cut would keep 79 % of the probe sets in 15 cells; the rule keeps the single root cell, and dominance lookups still catch the symmetric repeats.

8.8 Correctness

Theorem 8.4 (orbital root)**.** *For every probe set P of size d there is a group element valid on P that maps P into a cell of the orbital root. Consequently, if every probe set of the orbital root is secure, every probe set is.*

Proof sketch. By induction along the branching tree. At a node with group K, prefix *pre* (fixed pointwise by K), wire set F and count c, let Q be the part of the probe set still to place. If the node is kept whole the claim is trivial. If Q meets the stable domain D in a non-trivial orbit, let O_i be the first such orbit it meets; an element of K maps a wire of $Q \cap O_i$ to x_i (Lemma 8.1); K preserves each orbit, so the image still avoids $O_1, \ldots, O_{i-1}$ and lies in the i-th child; recurse with $\mathrm{Stab}(x_i)$, composing the elements (valid on D). If $Q \cap D$ lies in fixed wires, the fixed-part cells contain it. If Q touches loose wires, the loose-wire cases enumerate all possibilities, using subgroups of K. Every element used fixes *pre*. The second sentence follows from Theorem 6.2. □

The implementation checks the first statement by brute force on every gadget whose size permits it: every probe set must be mapped into a root cell by some listed element (`rootCheck` (`Symmetry.lean`), part of `lake exe verif golden`).

Orbital branching and dominance are complementary. Orbital branching decides, before the search starts and without looking at any probe set, which blocks need to be searched at all; it cannot see the duplicates that remain inside the blocks (Example 8.2). Dominance sees exactly those, one seed at a time, but only after the search has reached them. Measured on Keccak χ with 4 shares: dominance alone 414 ms, both 130 ms.

9 The complete algorithm

Algorithm 2 Symmetric cell search (`checkDProbingSym` (`Symmetry.lean`))

Require: circuit with body wires W, order d

1: $G \leftarrow$ relabellings of the atoms, found on the chain-contracted netlist (Section 6.3), closed and bounded by B; trivial if (S1) or (S2) fails
2: $H \leftarrow$ all non-identity elements of G, as forward and inverse wire tables
3: roots $\leftarrow$ the orbital root of G
4: stack $\leftarrow$ roots; $\mathcal{S} \leftarrow \emptyset$
5: **while** stack not empty **do**
6: pop a cell and visit it (Algorithm 1); on INSECURE stop
7: push its children
8: **return** SECURE

Theorem 9.1. *Algorithm 2 terminates. If it returns SECURE, the circuit is d-probing secure.*

Proof. Every probe set of the roots ends inside a stored safe set (Theorem 5.3); stored sets are safe (Theorem 4.1, Lemma 7.1); every probe set has an image in the roots (Theorem 8.4) and security transports along it (Theorem 6.2). □

10 Worked example: DOM-AND with 3 shares at order 2

The gadget has 21 body wires: products $u_i = a_i b_i$ and $p_{ij} = a_i b_j$, masked cross terms $c_{ij} = p_{ij} \oplus r_{ij}$ (with c_{ji} using the same r_{ij}), and outputs built as chains $s_i m_0 = u_i \oplus c_{i\cdot}$, $s_i = s_i m_0 \oplus c_{i\cdot}$. There are $\binom{21}{2} = 210$ pairs. The output below is from `lake exe verif trace "DOM AND (3 shares)"`.

Symmetry. A group of order $36 = 3! \cdot 3!$ (the shares of a and the shares of b permuted independently; most elements are valid on only part of the wires); 35 distinct non-identity automorphisms for the lookups. A typical element swaps share indices 0 and 1: $u_0 \leftrightarrow u_1$, $p_{01} \leftrightarrow p_{10}$, $c_{02} \leftrightarrow c_{12}$, $s_0 \leftrightarrow s_1$, …, and is not valid on $s_2 m_0$ (the chain $u_2 \oplus c_{20}$ would become $u_2 \oplus c_{21}$, which no wire computes).

Orbital root: 62 of 210 pairs in 9 cells.

cell	pairs
$\{u_0, u_1\}$, $\{u_0, p_{01}\}$, $\{u_0, p_{10}\}$, $\{u_0, s_1\}$, $\{u_0, s_0\}$, $\{s_0, s_1\}$	6×1
$\langle 1 : \{c_{01}\}\rangle \times \langle 1 : \{c_{10}, c_{02}, c_{20}, c_{12}, c_{21}, u_*, p_*, s_*\}\rangle$	17
$\langle 1 : \{s_0 m_0\}\rangle \times \langle 1 : \{s_1 m_0, u_*, p_*, c_*, s_*\}\rangle$	19
$\langle 1 : \{s_2 m_0\}\rangle \times \langle 1 : \{\text{all other 20 wires}\}\rangle$	20

The first six cover the 66 pairs inside the stable domain $\{u_*, p_*, s_*\}$ (the 12 wires on which every element is valid): the branching pinned u_0 (orbit of the products), then pinned the second wire within the orbits of $\mathrm{Stab}(u_0)$; the pair $\{s_0, s_1\}$ comes from the next orbit, the outputs. The c_{ij} are loose for the whole group, but a subgroup permutes them, so every pair containing a masked term is represented by a pair containing c_{01}. The chain intermediates are the least symmetric: $s_2 m_0$ gets its own pinned cell, whose further cutting does not pay (20 pairs, kept whole); $s_0 m_0$ and $s_1 m_0$ are exchanged by the group, so pairs with $s_1 m_0$ are represented in the $s_0 m_0$ cell.

The search: 28 cell visits, 9 engine calls.

step	what happens
1	cell $\{u_0, u_1\}$: engine certifies it; extension gives $\{u_0, u_1, p_{01}, p_{10}, c_{01}, c_{10}, s_0m_0, s_1m_0\}$, stored.
2–3	$\{u_0, p_{01}\}$ and $\{u_0, p_{10}\}$: skipped, inside that set.
4	$\{u_0, s_1\}$: engine; stores $\{u_0, s_1, u_1, p_{01}, p_{10}\}$.
5	$\{u_0, s_0\}$: *dominance* — the relabelling that cycles the shares of a ($a_0 \mapsto a_2 \mapsto a_1 \mapsto a_0$) and leaves those of b alone maps $\{p_{10}, s_1\}$, inside the set of step 4, to $\{u_0, s_0\}$; the image $\{u_0, p_{01}, p_{20}, p_{21}, s_0\}$ is stored. No engine call.
6	$\{s_0, s_1\}$: engine; stores $\{s_0, s_1, u_0, u_1, p_{01}, p_{10}\}$.
7	the c_{01} cell (17 pairs): its seed $\{c_{01}, c_{10}\}$ is inside the set of step 1; split into children.
8–14	the c_{01} children: 3 engine calls, 4 dominance hits.
15–21	the s_0m_0 cell: 2 engine calls, 2 dominance hits, 2 covers, 1 skip.
22–28	the s_2m_0 cell: 1 engine call, 5 dominance hits, 1 cover.
total	9 engine calls, 16 covers (12 of them through symmetry), 3 skips, 21 stored sets.

For comparison: without symmetry the same search makes 30 engine calls and 68 cell visits; the split search makes 84 engine calls (including union attempts); maskVerif checks the 435 pairs of its own, larger observation set.

11 How the implementation is checked

Every change on the branch must leave the following check unchanged (176 lines):

```
lake exe verif golden check results/engine-golden_baseline.txt
```

It records:

- for every gadget, the engine's verdict, coupling and extension on 100–300 seeded random probe sets, in a node-id-independent form (detects any change in the engine);
- the end-to-end result of the split search, the cell search and the symmetric search;
- **coverage audit**: after a secure run, every probe set of the starting cells lies inside a stored safe set (`coverageAudit`, `coverageAuditCells`) — checked by enumeration, e.g. all 1 873 200 triples of Keccak χ with 4 shares;
- **root audit**: every probe set is mapped into a root cell by a listed element (`rootCheck`); skipped only for the two largest gadgets (budget);
- the one-pass `contractAssoc` against its step-by-step definition (`golden contract`).

One further check, `bench transport`, is run by hand because it is slow. It tests the property that all uses of symmetry rest on, sharing nothing with the code under test: it evaluates the raw netlist on every assignment of the leaves, with no DAG and no interning, and compares the conditional distributions of a probe set and of its image. It covers the 38 entries with at most 18 leaves, at probe sizes d and $d+1$ (so that insecure sets are included), enumerating where the budget allows and sampling otherwise:

- all listed group elements: 99 254 probe sets, no violation;
- free relabellings (swaps of two shares of a sharing, of two gadget randoms, or of two whole sharings, from no search at all, as in Theorem 6.6): 5 487 non-trivial probe sets, no violation;

- corrupted tables (two images exchanged): caught in 84 of 101 cases, so the check has teeth; the other 17 went undetected on the probe sets tried, for instance because the exchanged images have the same distribution;
- the swap $a_1 \leftrightarrow b_1$ of Remark 6.4: the wire map $v \mapsto w$ is produced and the distributions differ, as predicted.

The five gadgets with more than 18 leaves are out of reach of this check. The two audits re-check, independently of the search's own bookkeeping, the two facts its soundness rests on (Theorems 5.3 and 8.4). Safety of each stored set itself rests on the engine and Theorems 4.1 and 6.2.

12 Measurements

All figures are from 31 October 2026, except the maskVerif column below (28 October), on one desktop machine under light background load (±20 %); details in `lean_ffi/verif/results/`.

What each idea contributes (milliseconds, in process, setup included):

	split search	cells, no symmetry	cells + dominance only	full (Alg. 2)
41-entry suite (sum)	3 271	126	—	131
DOM Keccak χ, 4 shares, $d = 3$	> 25 min*	1 885	414	130
DOM-AND, 5 shares, $d = 4$	—	110	84	65
ISW-AND, 5 shares, $d = 4$	563	59	—	66

* before the engine changes.

The group bound. Groups of order 576 and more (DOM-AND with 4 and 5 shares) cost more to list, tabulate, branch on and look up through than they save. DOM-AND-4 takes 47 ms at bound 600, where its whole group fits, and 7 ms at 64, for one more engine call; DOM-AND-5 takes 104 ms at 600 and 65 ms at 64. Keccak χ with 4 shares (order 30) is unaffected at 64 and takes 60 % longer at 24.

Against maskVerif (one process per gadget, best of 3 to 5, each tool's start-up time subtracted; `results/maskverif-comparison_2026-10-28.md`):

gadget	maskVerif [ms]	this tool [ms]
DOM Keccak χ, 4 shares, $d = 3$	426	127
DOM-AND, 5 shares, $d = 4$	371	69
ISW-AND, 5 shares, $d = 4$	213	66
DOM-AND, 4 shares, $d = 3$	27	8
DOM Keccak χ, 3 shares, $d = 2$	18	8
TI Fides AB1, $d = 1$	12	26
33 secure suite entries (sum)	331	171

The verdicts agree on all 44 gadgets. maskVerif [3] also probes input shares and randoms (275 against 225 positions on Keccak χ with 4 shares).

13 Limitations and open questions

- **Fixed parameters.** The group bound 64 and the "at least a quarter" rule were measured on this suite only.

- **Not one representative per orbit.** The root leaves duplicates whenever a probe set meets its first orbit twice (Example 8.2) or the stabilisers are small; dominance removes them later, at the cost of a lookup.
- **Inputs outside (S1) and (S2).** Such a circuit gets no symmetry, which is sound but slower. One entry of the suite is affected, *xy TI virtual variable*, whose inputs z_1, z_2, z_3 are three fresh randoms with no secret, which `sharingsOf` does not accept as a sharing (6 engine calls instead of 5). Renaming raw leaves (Section 6.4) needs neither condition and could be added as a fallback.
- **Choice of subgroup.** Above the bound B the group is reduced greedily. For DOM-AND with 4 shares this gives order 24, although the carrier-keeping subgroup of order 36 also fits and gives a slightly smaller root (3 461 against 3 611 probe sets, 35 against 38 engine calls, no gain in time).
- **Partial symmetry.** The loose-wire construction is a heuristic choice of subgroups; its correctness is checked by `rootCheck`, not proved in Lean.
- **TI Fides AB1.** Its couplings relabel the input-encoding randoms, which almost every wire depends on, so nearly every candidate needs the slow path of the extension. It also has no usable symmetry: the wire tables of its group of order 2 move no wire.
- **Model.** Value-based probing only; glitches and transitions are out of scope. Symmetry is cheaper to use in this model, because two wires with the same value function are interchangeable.
- **Proof status.** Theorem 4.1 is argued in the design memo, and Theorems 6.2 and 6.6 and Lemmas 6.8 and 6.9 in these notes; the Lean formalisation on the `formally-verified` branch covers the engine and the split search, not the cell search or the symmetry.

References

[1] Y. Ishai, A. Sahai, D. Wagner. Private circuits: securing hardware against probing attacks. CRYPTO 2003.

[2] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub. Verified proofs of higher-order masking. EUROCRYPT 2015.

[3] G. Barthe, S. Belaïd, G. Cassiers, P.-A. Fouque, B. Grégoire, F.-X. Standaert. maskVerif: automated verification of higher-order masking in presence of physical defaults. ESORICS 2019.

[4] J. Ostrowski, J. Linderoth, F. Rossi, S. Smriglio. Orbital branching. Mathematical Programming 126(1), 2011.

[5] F. Margot. Pruning by isomorphism in branch-and-cut. Mathematical Programming 94, 2002.

[6] T. Fahle, S. Schamberger, M. Sellmann. Symmetry breaking. CP 2001.

[7] F. Focacci, M. Milano. Global cut framework for removing symmetries. CP 2001.

[8] B. D. McKay. Isomorph-free exhaustive generation. Journal of Algorithms 26, 1998.

[9] B. D. McKay, A. Piperno. Practical graph isomorphism, II. Journal of Symbolic Computation 60, 2014.